

CODE EASY ACADEMY

JS MASTERY MARATHON · WEEK 05

Objects & State Thinking

Stop managing variables. Start managing state.

INSIDE THIS PACK

- 01 — **The Mental Model** · What objects are and why they change everything
- 02 — **State Thinking** · Variables vs State — the shift that matters
- 03 — **Live Demo — Shopping Cart** · A real app, built piece by piece
- 04 — **Common Bugs** · The 4 mistakes that will bite you
- 05 — **Exercise Pack** · 10 Problems
- 06 — **Self-check** · Before you submit

THE PROMISE THIS WEEK

By the end of this session, you'll stop thinking in scattered variables and start thinking in state — one object that holds everything your app needs to know. This is the shift from "I can write JavaScript" to "I can model a real app."

Submissions due Friday night.

PART 01 · THE MENTAL MODEL

The problem we're solving first. You've been doing this:

```
let userName = "Aman";
let userAge  = 22;
let userCity = "Bhopal";
// ... 30 users = 30 variables. Disaster.
```

Objects fix this:

```
let user = {
  name:    "Aman",
  age:     22,
  city:    "Bhopal",
  isLoggedIn: true
};
```

One variable. Everything about that user lives inside it. This is an **object** — a container that groups related data under one name.

The anatomy of an object

```
let product = {  
  name:    "Shoes",    // key: value  
  price:   2000,  
  inStock: true  
};
```

- The things on the left (`name` , `price` , `inStock`) are called **keys**
- The things on the right are the **values**
- Together, each pair is called a **property**

Reading a property — dot notation

```
console.log(product.name);    // Shoes  
console.log(product.price);   // 2000  
console.log(product.inStock); // true
```

Updating a property

```
product.price    = 1800;    // price changed  
product.inStock = false;    // now out of stock
```

Adding a new property

```
product.discount = 10;    // didn't exist before -- now it does  
console.log(product.discount); // 10
```

THE ANALOGY

Think of an object like a form. A Zomato order form has fields: restaurant name, items, address, total, status. Each field has a value. The form is the object. The fields are the keys. When your order status changes from "Preparing" to "Out for Delivery" — Zomato isn't creating a new form. They're updating one field on the same form. That's state.

PART 02 · STATE THINKING

What is state?

State is what your app remembers *right now*. Not what it did yesterday. Not what it will do tomorrow. What it knows at this precise moment.

Variables vs State

```
// Variables -- scattered, unconnected
let item1      = "Shoes";
let item1Price = 2000;
let item2      = "T-Shirt";
let item2Price = 800;
let total      = 2800;
let couponApplied = false;
```

This is 6 separate variables. Nothing connects them. If you add an item, you have to remember to update `total` manually. If you forget — your app shows the wrong total. This is how bugs are born.

```
// State -- one object, everything connected
let cart = {
  items:      [],
  total:      0,
  couponApplied: false
};
```

THE MENTAL MODEL FOR WEEK 5

Your app doesn't have variables. Your app has **state**. State lives in an object. Functions read and update that state.

PART 03 · LIVE DEMO — SHOPPING CART

We build this together. Step by step. Don't skip ahead.

Step 1 · Define the state

Before writing a single function — define the state. What does the cart need to know? Items, total, count. That's it.

```
let cart = {
  items:      [],      // array of items added
  total:      0,        // running total in Rs.
  itemCount:  0,        // how many items
  couponApplied: false
};
```

Step 2 · A function to add an item

```
function addItem(name, price) {  
  let newItem = { name: name, price: price };  
  cart.items.push(newItem);  
  cart.total    = cart.total + price;  
  cart.itemCount = cart.itemCount + 1;  
  console.log(name + " added to cart.");  
}
```

The function doesn't return anything. It updates the state object directly. That's intentional — the state object IS the source of truth.

Step 3 · A function to show the cart

```
function showCart() {  
  console.log("--- YOUR CART ---");  
  if (cart.itemCount === 0) {  
    console.log("Cart is empty.");  
    return;  
  }  
  for (let i = 0; i < cart.items.length; i++) {  
    let item = cart.items[i];  
    console.log(item.name + ": Rs." + item.price);  
  }  
  console.log("Total: Rs." + cart.total);  
}
```

THE LOOP MOMENT: `cart.items` is an array of objects. Each item inside it is an object with `.name` and `.price`. Arrays of objects — arrived naturally, not as a new concept to memorise.

Step 4 · Remove the last item

```
function removeLastItem() {  
  if (cart.itemCount === 0) {  
    console.log("Nothing to remove.");  
    return;  
  }  
  let removed = cart.items.pop();  
  cart.total   = cart.total - removed.price;  
  cart.itemCount = cart.itemCount - 1;  
  console.log(removed.name + " removed from cart.");  
}
```

Step 5 · Apply a coupon

```
function applyCoupon(discountPercent) {
  if (cart.couponApplied) {
    console.log("Coupon already applied.");
    return;
  }
  if (cart.itemCount === 0) {
    console.log("Add items before applying a coupon.");
    return;
  }
  let discount      = (cart.total * discountPercent) / 100;
  cart.total        = cart.total - discount;
  cart.couponApplied = true;
  console.log("Coupon applied! You saved Rs." + discount);
}
```

Step 6 · Run the cart

```
addItem("Shoes", 2000);
addItem("T-Shirt", 800);
addItem("Cap", 400);
showCart();
applyCoupon(10);
showCart();
removeLastItem();
showCart();
```

NOTICE: We never created a new cart. We never reset the variables. We just updated one object — and every function that reads from it automatically got the right numbers. This is state. This is how Swiggy tracks your order. This is how Spotify knows which song is playing.

PART 04 · COMMON BUGS

The 4 mistakes that will cost you time this week.

Bug 1 · Accessing a key that doesn't exist

```
let user = { name: "Aman", age: 22 };
console.log(user.city); // undefined -- not an error, just undefined
```

JavaScript won't throw an error — it gives you `undefined` silently. This is dangerous because `undefined` can break your logic later. Always check what keys your object actually has before reading them.

Bug 2 · Forgetting to update the state object

```
// WRONG -- creates a local variable, doesn't update cart
function addPrice(price) {
  let total = cart.total + price;
}

// RIGHT
function addPrice(price) {
  cart.total = cart.total + price;
}
```

Bug 3 · Reassigning the whole object

```
let product = { name: "Shoes", price: 2000 };

// WRONG -- wipes all other properties
product = { name: "Shirt" }; // product.price is now gone

// RIGHT -- update individual properties
product.name = "Shirt";      // product.price still exists
```

Use dot notation to update individual properties. Don't reassign the whole object unless you mean to wipe it.

Bug 4 · Looping over objects and forgetting the dot

```
for (let i = 0; i < items.length; i++) {
  console.log(items[i]);           // prints the whole object
  console.log(items[i].name);      // THIS is what you want
  console.log(items[i].price);     // and this
}
```

`items[i]` gives you the object. `items[i].name` gives you the value inside it.

PART 05 · EXERCISE PACK

10 problems · 2 Easy · 5 Medium · 3 Challenge · + Bonus

HOW TO TACKLE EACH PROBLEM

1. **Define your state object FIRST** — what does it need to know?
2. **Decompose** — Inputs, Process, Output, Edges in plain English
3. **Pseudocode** — plain English steps first
4. **JavaScript** — only now open the editor

PROBLEM 01 ★ EASY — Student Profile

Create an object called `student` with these properties: `name`, `age`, `city`, `course`, `isEnrolled`. Then write code that:

- Prints each property using dot notation
- Updates the city to "Mumbai"
- Adds a new property `batchNumber` with value 4
- Prints the updated object

EDGES: What happens if you try to read a property that doesn't exist?

PROBLEM 02 ★ EASY — Product Card

Create an object `product` for a phone:

- `name`: "Redmi Note 13"
- `price`: 15999
- `inStock`: true
- `rating`: 4.3

Write a function `showProductCard(product)` that prints the name, price, rating and stock status cleanly.

EDGES: `inStock` is false (print "Out of Stock"), rating above 5

PROBLEM 03 ★★ MEDIUM — Bank Account

Model a bank account as a state object:

```
let account = { owner: "Priya", balance: 5000, transactionCount: 0 }
```

Write three functions:

- `deposit(amount)` — adds to balance, increments `transactionCount`
- `withdraw(amount)` — subtracts from balance (don't allow going below 0), increments `transactionCount`
- `showBalance()` — prints owner name, current balance, total transactions

EDGES: Withdraw more than balance, deposit 0 or negative, withdraw 0

PROBLEM 04 ★★ MEDIUM — Movie Watchlist

Create a state object:

```
let watchlist = { movies: [], totalAdded: 0 }
```

Write:

- `addMovie(title, genre)` — adds a movie object to the list
- `showWatchlist()` — prints each movie's title and genre
- `countByGenre(genre)` — returns how many movies match that genre

EDGES: Empty watchlist, genre with no matches, duplicate titles

PROBLEM 05 ★★ MEDIUM — Zomato Order Tracker

Model an order as a state object. An order has: `restaurant`, `items` (array), `status`, `total`.

Write:

- `placeOrder(restaurant, items, total)` — sets up the order, status = "Placed"
- `updateStatus(newStatus)` — updates the status
- `showOrder()` — prints all order details cleanly

Valid statuses: "Placed" → "Confirmed" → "Preparing" → "Out for Delivery" → "Delivered"

EDGES: Updating to an invalid status, showing order before placing it

PROBLEM 06 ★★ MEDIUM — Quiz Score Tracker

```
let quiz = { playerName: "", score: 0, questionsAttempted: 0, correctAnswers: 0 }
```

Write:

- `startQuiz(name)` — sets the player name, resets all scores
- `submitAnswer(isCorrect)` — increments questionsAttempted; if correct, increments correctAnswers and adds 10 to score
- `showResult()` — prints name, score, accuracy percentage

EDGES: 0 questions attempted (don't divide by zero), all wrong, all correct

PROBLEM 07 ★★ MEDIUM — Inventory Manager

```
let inventory = { items: [], totalValue: 0 }
```

Write:

- `addItem(name, price, quantity)` — adds item object, updates totalValue
- `showInventory()` — prints each item: name, price, quantity, item total (price × quantity)
- `findItem(name)` — returns the item object if found, or `null` if not

EDGES: Item not found, quantity = 0, same item added twice (treat as separate entries)

PROBLEM 08 ★★★ CHALLENGE — Leaderboard

```
let leaderboard = { players: [], topScore: 0, topPlayer: "" }
```

Write:

- `addPlayer(name, score)` — adds player object, updates `topScore` and `topPlayer` if this score is highest
- `showLeaderboard()` — prints all players ranked by score (highest first) with rank number
- `getTopThree()` — returns an array of the top 3 player objects

EDGES: Tie scores, fewer than 3 players when calling `getTopThree`, same player added twice

HINT: For ranking, compare two scores and rearrange manually — same loop patterns from Week 3.

PROBLEM 09 ★★★ CHALLENGE · STAR PROBLEM

Shopping Cart — Full Build

This is the demo, extended. You've seen the pieces. Now build the whole thing from scratch.

```
let cart = { items: [], total: 0, itemCount: 0, couponApplied: false }
```

Write ALL of these:

- `addItem(name, price)` — adds item, updates total and count
- `removeItem(name)` — finds and removes item by name, updates total and count
- `applyCoupon(discountPercent)` — only once, only if cart isn't empty
- `showCart()` — clean receipt with all items and final total
- `clearCart()` — resets everything back to initial empty state

EDGES: Remove item that doesn't exist, apply coupon twice, clear an empty cart, show cart when empty

This is the one. Do not skip this.

PROBLEM 10 ★★★ CHALLENGE — Expense Splitter

```
let trip = { name: "Goa Trip", members: [], expenses: [], totalSpent: 0 }
```

Write:

- `addMember(name)` — adds a person to the trip
- `addExpense(paidBy, description, amount)` — logs an expense, updates `totalSpent`
- `showSummary()` — prints total spent, each expense, and how much each person owes (totalSpent / number of members)

EDGES: No members added, one member, member not in the list pays an expense

BONUS CHALLENGE

Pick any app you use daily — Spotify, Instagram, a food delivery app. Model its core state as a JavaScript object. Write at least 3 functions that read or update that state. There's no single right answer — the point is to think: what does this app need to remember?

Share your object design in the WhatsApp group before Friday. We'll pick the most interesting ones to walk through in Logic Lab.

PART 06 · SELF-CHECK BEFORE YOU SUBMIT

- ☐ Did I define my state object before writing any functions?
- ☐ Do my functions update the state object directly (using dot notation) instead of creating new local variables?
- ☐ Did I handle the edge cases listed in each problem?
- ☐ For the Star Problem — did I build it from scratch, without looking at the demo?
- ☐ Can I explain in one sentence: what state does this app hold?

IF YOU ONLY DO ONE THING

Do Problem 9. Rebuild the cart from scratch. If you can do that without looking at the demo — you've installed the mental model. That's the whole point of this week.

Post your code to the WhatsApp group by Friday night. We solve them together at Logic Lab.

Stuck on Problem 8 or 9? Go back to the demo. Trace through each function step by step. The answer is already in your notes — you just need to trust the framework.

See you Friday for Logic Lab.

— Rajdeep